# Computer Graphics

<TheAviator>

담당 교수 : 김동준 교수님

학번 : 2021204010

학과 : 정보융합학부

이름 : 박현준

# 목차

**1.** Transformer ---------- (3)

**2.** Scene Composition Overview ---------- (5)

**3.** Smooth Camera Transition Behind Airplane (Translation & Rotation Animation) ---------- (9)

**4.** Attach Camera: Convert World Coordinates to Airplane Local Coordinates ---------- (15)

**5.** Smooth Camera Movement During View Switch and Airplane Control ---------- (13)

**6.** Fixing Disconnected Parts in Sea Geometry ---------- (33)

# 1. Transformer

3D 컴퓨터 그래픽스에서 객체(Object)의 위치(Position), 방향(Rotation), 크기(Scale)등을 표현하고 제어하기 위해서는 **공간 변환(Spatial Transformation)**개념이 필수적이다. 모든 3차원 객체는 특정 위치에 존재하며, 일정한 방향을 향하고 있고, 고유한 스케일 값을 지니고 있다. 이러한 속성들을 수학적으로 명확하게 정의하고 연산 가능하게 만들기 위해 공간 변환은 필연적으로 요구된다.

이러한 변환 과정을 수학적으로 구현하는 구조체가 바로 **Transformer**이다. Transformer는 **위치 이동(Translation), 회전(Rotation), 크기 조정(Scaling)** 등 다양한 변환을 하나의 구조로 통합하며, 보통 4×4 형태의 **동차 좌표계(Homogeneous Coordinates)**기반 행렬로 표현된다. 이 하나의 **Transformer**를 통해 객체의 복합적인 공간 상태를 효율적으로 제어할 수 있다.

각 변환은 수학적으로 다음과 같이 정의된다.

- **회전(Rotation)**: 특정 축(axis)을 기준으로 한 각도 기반의 행렬 연산을 통해 구현된다.
- **이동(Translation)**: 위치 벡터를 추가하는 방식으로 적용된다.
- **크기 조정(Scale)**: 각 축 방향에 대해 비례적으로 값을 조정하는 방식으로 적용된다.

이러한 개별 Transformer들은 행렬 간 곱셈을 통해 하나의 복합 Transformer로 결합될 수 있으며, 이때 변환의 **순서**에 따라 결과가 달라지므로 적용 순서는 매우 중요한 의미를 가진다.

Transformer는 단순한 기하학적 조작 이상의 의미를 지닌다. 객체 간의 상대적 위치 관계를 정의하는 **계층적 구조(hierarchy)**, 장면 전체의 구조를 구성하는 장면 그래프(scene graph), 그리고 카메라의 시점 전환이나 이동과 같은 시각적 효과 구현에서도 필수적인 요소로 작용한다. 이처럼 Transformer는 3D 그래픽스 시스템 전반에서 객체의 공간적 존재 방식과 동작 원리를 결정짓는 핵심 수단이라 할 수 있다.

자세한 식이나 과정은 뒤에서 코드 설명과 함께 다루겠다. 다음으로 **World Space**와 **Local Space**, **World Transformer**와 **Local Transformer**의 개념을 간단히 설명하고 본론으로 들어가겠다.

## Local Space, World Space / Local Transformer, World Transformer

컴퓨터 그래픽스 시스템에서는 동일한 객체라도 관찰 기준이 되는 좌표계(Frame of Reference)에 따라 그 위치와 방향이 다르게 해석된다. 이를 설명하는 핵심 개념이 바로 **Local Space(로컬 좌표계)**와 **World Space(월드 좌표계)**이다.

**Local Space**는 객체 자체의 중심을 원점으로 하는 좌표계로, 해당 객체의 정점, 노멀 등 기하학적 정보는 이 기준에 따라 정의된다. 반면, **World Space**는 장면(Scene)의 전역 기준 좌표계이며, 최종적으로 렌더링되는 위치는 World Space 상의 좌표로 결정된다.

이 두 공간 간의 변환은 각각의 Transformer를 통해 이루어지며, 이를 **Local Transformer**와 **World Transformer**로 구분한다.

- **Local Transformer**는 해당 객체의 로컬 좌표계 내에서의 위치, 회전, 스케일을 정의한다.

- **World Transformer**는 해당 객체가 장면 전체(World 기준)에서 어디에 존재하고 어떤 방향을 바라보는지를 정의하며,, 계층적 구조에서는 다음과 같이 정의된다:

**[ World Transformer = Parent World Transformer × Local Transformer ]**

이를 통해 객체 간 계층적 관계가 형성된다. 이는 장면 그래프(Scene Graph)를 구성하는 핵심 원리이며, 객체가 부모에 종속되어 있을 경우 부모가 움직이면 자식 객체도 함께 이동하는 동작을 가능하게 한다.

## 2. Scene Composition Overview

### 1. Scene

```
function createScene() {

  HEIGHT = window.innerHeight;
  WIDTH = window.innerWidth;

  scene = new THREE.Scene();
```

· 역할: 모든 3D 객체가 포함되는 컨테이너 역할을 한다.

· 특징: Light, Mesh, Camera, Particle 등 모든 요소는 이 Scene 객체에 추가되어야 rendering이 가능하다.

### 2. Camera

```
camera = new THREE.PerspectiveCamera(
  fieldOfView,
  aspectRatio,
  nearPlane,
  farPlane
);
//scene.fog = new THREE.Fog(0xf7d9aa, 100, 950);
camera.position.x = 0;
camera.position.z = 200;
camera.position.y = game.planeDefaultHeight;
```

· 역할: 3D 공간을 관찰하는 시점을 정의한다.

· 속성

  · fieldOfView: 시야각 (여기서는 50도).

  · aspectRatio: 화면의 종횡비로, window.innerWidth / window.innerHeight값으로 설정.

  · nearPlane, farPlane: 카메라가 렌더링할 수 있는 거리 범위 (0.1~10000 단위).

· 초기 위치: 비행기의 높이에 맞게 (0, game.planeDefaultHeight, 200)로 설정

### 3. Light

```
var ambientLight, hemisphereLight, shadowLight;

function createLights() {

  hemisphereLight = new THREE.HemisphereLight(0xaaaaaa, 0x000000, .9)

  ambientLight = new THREE.AmbientLight(0xdc8874, 1);

  shadowLight = new THREE.DirectionalLight(0xffffff, 5.9);
```

Scene에는 총 세 가지 조명이 사용되어 물체의 입체감과 분위기를 자연스럽게 표현한다.

· **HemisphereLight:** 하늘과 지면의 색을 다르게 비추는 반구형 조명으로, 부드러운 환경광을 연출한다.

· **AmbientLight:** 모든 방향에서 균일하게 비추는 조명으로, 장면 전체의 기본 밝기를 설정한다.

· **DirectionalLight(변수명: shadowLight):** 태양빛처럼 한 방향으로 강하게 비추는 직사광. 그림자를 생성하며, 그림자의 범위와 품질을 세밀하게 조정해 현실감을 높인다.

### 4. Pilot

```
var Pilot = function () {
  this.mesh = new THREE.Object3D();
  this.mesh.name = "pilot";
```

Pilot은 다음과 같은 요소로 구성된다:

· Body, Face, Hair, Glasss, Ears

각 요소는 모두 THREE.Mesh로 구성되어 있으며, 하나의 Object3D에 결합되어 pilot모델을 형성한다.

### 5. Plane

```
var AirPlane = function () {
  this.mesh = new THREE.Object3D();
  this.mesh.name = "airPlane";
```

비행기는 다양한 부품으로 구성된 하나의 복합 객체로, 다음과 같은 주요 요소들로 구성된다:

· Cabin, Engine, Tail Plane, Side Wing, Wind Shield, Propeller(blade1,2), WheelProtecR(L), WheelTire(R,L,B), Wheel Axis, Suspension), Pilot

Pilot은 **this.mesh.add(this.pilot.mesh);** 를 통해 plane의 자식으로 추가된다. 그 결과, plane이 움직일 때, pilot도 함께 움직이는 구조가 만들어진다.

**6. Sky**

```
function Sky() {
  this.mesh = new THREE.Object3D();
  this.nClouds = 20;
```

Sky는 20개의 Cloud인스턴스로 구성된 하늘 배경 요소다. 각 구름은 원형으로 분포되도록 x, y좌표를 삼각함수로 계산하지만, z좌표가 무작위로 설정되어 시각적으로는 사각형에 가까운 형태로 보인다. Sky 전체는 Z축을 중심으로 회전하며, 구름들이 천천히 움직이는 동적인 하늘 연출을 제공한다.

**7. Sea**

```
function Sea() {
  // 기본 geometry 생성
  let geom = new THREE.CylinderGeometry(game.seaRadius, game.seaRadius, game.seaLength, 40, 10);
  geom.rotateX(-Math.PI / 2);
```

Sea는 CylinderGeometry를 기반으로 생성된 객체로, 바닥 평면을 표현한다. 정점 병합(mergeVertices)과 노멀 재계산(computeVertexNormals)을 통해 파도 애니메이션에 적합한 형태로 가공되며, 각 정점은 개별적인 파도 속성을 가진다. moveWaves()함수를 통해 정점의 x,y 좌표를 시간에 따라 진동시켜 출렁이는 듯한 효과를 연출한다.

**8. Cloud**

```
function Cloud() {
  this.mesh = new THREE.Object3D();
  this.mesh.name = "cloud";
```

Cloud는 여러 개의 큐브(20x20x20)를 무작위로 배치하고 회전시켜 하나의 구름처럼 보이도록 만든다. rotate()함수는 구름을 구성하는 큐브들을 조금씩 회전시켜 생동감 있는 효과를 만든다.

**9. Ennemy & EnnemiesHolder**

```
function Ennemy() {
  var geom = new THREE.TetrahedronGeometry(8, 2);
  var mat = new THREE.MeshPhongMaterial({
```

Ennemy는 TetrahedronGeometry(8, 2)를 사용해 정사면체 형태를 기반으로 하지만 detail level이 2로 설정되어 있어 시각적으로는 단순한 정사면체가 아니라 복잡하고 불규칙한, 돌멩이 같은 형태로 보인다.

```
function EnnemiesHolder() {
  this.mesh = new THREE.Object3D();
  this.ennemiesInUse = [];
}
```

EnnemiesHolder는 다수의 적을 생성하고 관리하며, spawnEnnemies()에서 적들을 씬에 배치한다.

rotateEnnemies()함수는 적들이 원형 경로를 따라 회전하게 하고, 플레이어와 충돌하면 입자 효과를 발생시키고 에너지를 감소시킨다.

**10. Particle & ParticlesHolder**

```
function Particle() {
  var geom = new THREE.TetrahedronGeometry(3, 0);
  var mat = new THREE.MeshPhongMaterial({
```

```
function ParticlesHolder() {
  this.mesh = new THREE.Object3D();
  this.particlesInUse = [];
}
```

Particle는 정사면체 형태의 입자를 생성하고, explode()메서드를 통해 위치, 색상, 크기를 설정한 후 TweenMax 애니메이션으로 회전·축소·이동하는 폭발 효과를 연출한다. 애니메이션이 끝난 후에는 재사용을 위해 파티클 풀에 반환된다.

ParticlesHolder는 파티클을 담는 컨테이너 역할을 하며, spawnParticles()메서드를 통해 여러 개의 파티클을 생성하고 폭발시키는 연출을 처리한다. 오브젝트 풀링 방식으로 파티클을 재활용해 성능을 최적화한다.

이처럼 자주 생성/제거되는 오브젝트에 대해 풀링 방식을 적용함으로써, GC(가비지 컬렉션) 비용을 줄이고 FPS 성능을 안정적으로 유지할 수 있다.

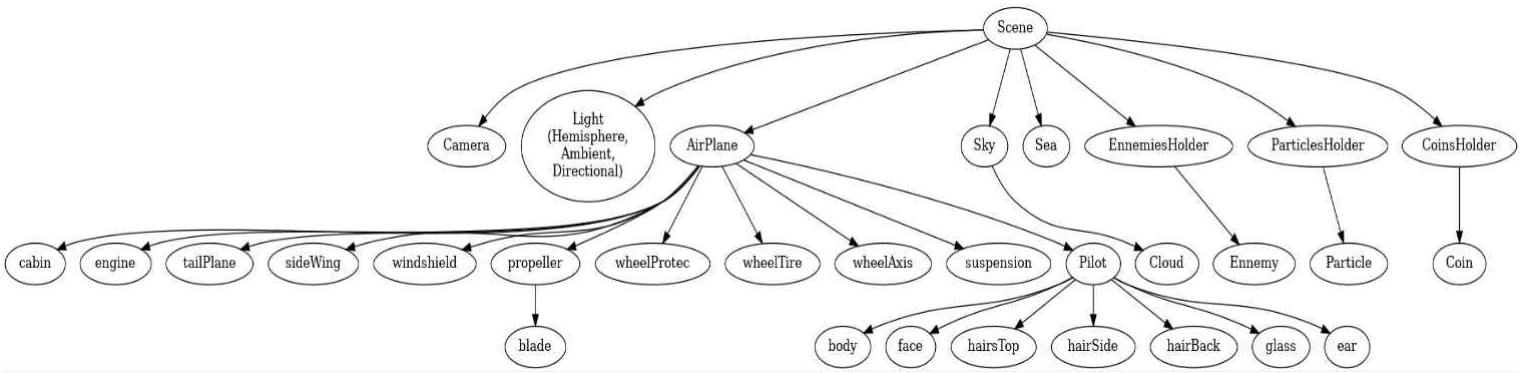
### 11. Coin & CoinsHolder(swapCoins(), rotationCoins())

```
function Coin() {
  var geom = new THREE.TetrahedronGeometry(5, 0);
  var mat = new THREE.MeshPhongMaterial({
```

```
function CoinsHolder(nCoins) {
  this.mesh = new THREE.Object3D();
  this.coinsInUse = [];
  this.coinsPool = [];
```

Coin은 정사면체 모양의 코인을 생성하고, 위치와 회전 정보를 관리한다. CoinsHolder는 이러한 코인들을 미리 풀(pool)로 저장하고, 필요할 때 꺼내어 장면에 배치한다. spawnCoins()메서드는 코인을 무작위 개수로 생성하여 원형 경로에 배치하고, rotateCoins()는 회전시키며 플레이어와 충돌 시 폭발 효과를 주고 에너지를 회복시키며 코인을 재활용한다. 이 구조는 퍼포먼스를 고려한 오브젝트 풀링 방식을 따른다.

이처럼 자주 생성/제거되는 오브젝트에 대해 풀링 방식을 적용함으로써, GC(가비지 컬렉션) 비용을 줄이고 FPS 성능을 안정적으로 유지할 수 있다.




[ 구조도 시각화 다이어그램 ]

# 3. Smooth Camera Transition Behind Airplane (Translation & Rotation Animation)

코드는 사용자가 space bar를 누르면 카메라를 비행기 뒤쪽으로 부드럽게 이동시키는 기능을 수행한다. 이 기능은 updateCameraTransition() 함수 안에서 매 프레임마다 호출되어 카메라의 위치(position)와 방향(Rotation)을 지속적으로 보간(Lerp,Slerp)하며 이동시킨다.

```
// 1인칭 시점: cockpit 위치 계산 (비행기 로컬 좌표계를 이용)
const cockpitOffset = new THREE.Vector3(-500, 25, 10);
// 비행기 로컬 좌표계에서 월드 좌표로 변환
const cockpitWorldPos = airplane.mesh.localToWorld(cockpitOffset.clone());
targetPos.copy(cockpitWorldPos);
```

### 3.1 비행기의 Local Space에서의 offset 설정

카메라를 비행기 뒤쪽에 배치하기 위해, 먼저 비행기의 Local Space에서 상대적인 위치를 나타내는 offset 벡터를 정의하였다.

**[ const cockpitOffset = new THREE.Vector3(-500, 25, 10); ]**

이 벡터는 비행기 기준(Local Space)으로 뒤쪽으로 500 단위, 위로 25 단위, 오른쪽으로 10 단위 떨어진 위치를 의미하며, 적절한 카메라 시점을 시각적으로 확인하며 반복 실험을 통해 결정된 값이다.

### 3.2 Local space to World space

Three.js의 카메라 위치는 World Space 기준으로 설정되어야 하므로, 이 로컬 기준의 offset 벡터는 World Space 좌표로 변환해야 한다. 이때 사용하는 메서드는 다음과 같다:

**[ const cockpitWorldPos = airplane.mesh.localToWorld(cockpitOffset.clone()); ]**

이 코드는 내부적으로 **비행기 객체의 World Transformer** $\mathbf{T_A}$를 이용하여 Local Space에서 정의된 벡터 $\mathbf{V_{local}}$ (cockpitOffset)를 World Space상의 위치 $\mathbf{V_{world}}$로 변환하는 작업을 수행한다. 이 수학적으로는 다음과 같이 표현된다.

$$\mathbf{V_{world}} = \mathbf{T_A} \cdot \mathbf{V_{local}}$$

즉, 비행기의 위치와 회전을 포함한 월드 변환 행렬을 적용함으로써, 비행기가 어느 위치에 있고 어떤 방향을 향하고 있든 항상 같은 상대 위치에 카메라가 있도록 할 수 있다.

이와 같은 변환 방식은 **3인칭 시점에서 비행기 뒤편으로 카메라를 전환할 때 비행기가 화면을 순간적으로 벗어나는 문제를 방지하고, 카메라가 자연스럽게 부드럽게 따라가도록 시점을 조정하는데 유용하다.** 이렇게 계산된 cockpitWorldPos는 최종적으로 카메라가 이동해야 할 목표 위치로 설정되며, targetPos에 복사된다.

<-- 비행기의 왼쪽 날개

[ World Space 상의 직접 위치 지정 방식의 결과 사진 - 3인칭에서 시점 변환 시 비행기가 우측 화면으로 사라졌다가 돌아옴 ]

[ 위 방식을 사용하여 수정한 결과 사진 - 시점 변환 시 비행기가 화면 밖으로 사라지지 않고 잘 진행됨]

```
const lookAtOffset = new THREE.Vector3(100, 0, 0);
const cockpitLookAt = airplane.mesh.localToWorld(lookAtOffset.clone());
targetLookAt.copy(cockpitLookAt);
```

카메라가 단순히 비행기 뒤에 위치하는 것뿐 아니라, 어디를 바라볼지도 정확하게 정의되어야 자연스럽고 몰입감 있는 시점이 완성된다. 이를 위해 코드에서는 다음과 같이 **Local Space 기준의 offset 벡터**를 하나 더 정의한다:

**[ const lookAtOffset = new THREE.Vector3(100, 0, 0); ]**

이 벡터는 비행기의 Local Space 기준으로 "앞쪽"을 향한 방향으로 100 단위떨어진 위치를 의미한다. 즉, 비행기 앞쪽을 향하는 방향을 기준으로 100만큼 떨어진 지점을 카메라가 바라보는 시점으로 삼겠다는 뜻이다.

이후, 이 로컬 좌표를 월드 좌표로 변환하여 실제 카메라가 바라보는 지점을 계산한다:

**[ const cockpitLookAt = airplane.mesh.localToWorld(lookAtOffset.clone()); ]**

**[ targetLookAt.copy(cockpitLookAt); ]**

비행기는 항상 움직이고 회전하는 객체이기 때문에, **World Space 기준으로 바라보는 방향을 고정**하면 시점이 어색하거나 엉뚱한 방향을 바라보게 될 수 있다. 따라서, 비행기의 현재 방향을 기준으로 "앞쪽" 방향을 설정하기 위해 Local Space에서 정의한 offset을 사용하였다.

이렇게 계산된 targetLookAt은 나중에 lookAt()함수의 기준점으로 활용되며, Matrix4.lookAt()과 Quaternion.slerp()를 통해 **카메라의 Rotation 방향을 부드럽게 보간**하는 데 사용된다.

### 3.3 카메라의 위치 보간 (Lerp를 통한 부드러운 이동)

카메라가 비행기의 뒤편으로 **부드럽게 이동**하도록 하기 위해, 다음과 같은 **선형 보간 연산**이 사용된다:

```
// 부드러운 위치 전환: 현재 카메라 위치에서 목표 위치로 보간(lerp)
camera.position.lerp(targetPos, 0.03);
```

이 코드는 camera.position을 목표 위치인 targetPos방향으로 조금씩 이동시키는 연산이다. 사용된 .lerp()는 THREE.Vector3의 메서드로, 다음과 같은 형식을 가진다:

Vector3.lerp(v: Vector3, alpha: Number)

v: 보간의 목표 벡터 (이 경우는 targetPos), alpha: 보간 비율 (0 ~ 1 사이의 값. 코드 상 0.03)

이 보간은 다음 수식으로 표현된다:

$$\mathbf{P}_{\text{new}} = (1 - \alpha) \cdot \mathbf{P}_{\text{current}} + \alpha \cdot \mathbf{P}_{\text{target}}$$

즉, 현재 위치에서 목표 위치로 향하는 벡터를 따라 일정 비율만큼 이동한 중간 위치를 계산해주는 방식이다. alpha값이 작을수록 천천히, 클수록 빠르게 목표에 접근한다.

**.lerp() 사용하는 이유**

카메라가 갑자기 위치를 바꾸면 화면 전환이 매우 부자연스럽고 시각적으로 어색하게보인다. 이를 방지하고자 .lerp()를 사용해 다음과 같은 효과를 얻는다:

· **부드러운 전환(Smooth transition)**: 갑작스러운 위치 변경 없이 자연스럽게 카메라가 이동한다.

· **안정적인 시점 유지(Natural tracking)**: 비행기를 천천히 따라가는 느낌을 주며, 몰입감을 높인다.

· **프레임 기반 반응(Frame-dependent update)**: 매 프레임마다 일정 비율만큼 이동하므로 FPS에

맞춰 자연스럽게 반응한다.

### 3.4 Camera Rotation Interpolation – .slerp()를 통한 Quaternion 기반 Rotation

```
const m = new THREE.Matrix4();
m.lookAt(camera.position, targetLookAt, camera.up);
const targetQuat = new THREE.Quaternion().setFromRotationMatrix(m);
camera.quaternion.slerp(targetQuat, 0.03);
```

비행기 뒤편으로 카메라가 이동할 때, 단순히 위치만 옮기는 것뿐만 아니라 시선 방향 또한 자연스럽게 회전하도록 설정해야 한다. 이를 위해 카메라가 특정 지점을 바라보도록 하는 회전 변환을 계산하고, **그 회전이 부드럽게 적용되도록 보간(slerp)**하는 과정이 필요하다.

#### 1) Rotation Matrix 생성: .lookAt()

**Matrix4.lookAt()**메서드는 Three.js에서 제공하는 함수로, 입력된 **camera.position(현재 카메라 위치)**에서 **targetLookAt(바라볼 목표 위치) 방향**을, camera.up을 기준으로 바라보도록 하는 **Rotation Matrix를** 생성한다.

이 행렬 m은 카메라의 **-z축이 바라보는 방향**, y축은 camera.up벡터, 그리고 x축은 이 둘의 외적(cross product)으로 구성되어 직교성(orthonormality)을 유지한 4×4 행렬이다.

#### 2) 쿼터니언 변환: .setFromRotationMatrix()

위에서 생성된 Rotation Matrix m을 회전을 표현하는 쿼터니언으로 변환한다. **Quaternion**은 3D 회전에서 **회전축 + 회전각정보를 하나의 4차원 벡터** 형태로 표현하며, **Gimbal Lock**이 발생하지 않고 부드러운 보간이 가능하다는 장점이 있다.

setFromRotationMatrix()는 행렬 m에 내포된 x, y, z축의 회전을 추출하여 **Quaternion** 값으로 매핑한다. 수학적으로는 행렬에서 쿼터니언 요소 $q_x$, $q_y$, $q_z$, $q_w$를 도출하는 변환식을 따르게 된다.

#### 3) 부드러운 Rotation Interpolation: .slerp()

**slerp()**는 현재 쿼터니언(camera.quaternion)에서 목표 **Quaternion**(targetQuat)으로의 회전을 곡선 경로 상에서 일정 비율만큼만 진행하는 함수이다.

인자 0.03는 매 프레임당 3%씩 목표 회전으로 이동하도록 설정한다. 이 값이 클수록 회전이 빨리 일어나고, 작을수록 천천히 회전한다.

slerp()는 일반적인 선형 보간(lerp)과 달리 **회전의 단위 구면 공간($S^3$) 상에서 가장 짧은 호(arc)를 따라 움직이는 방식**이므로, 회전의 자연스러움과 일관성이 매우 뛰어나다.

이 과정들을 통해, 카메라가 비행기 뒤편의 새로운 위치로 이동할 뿐만 아니라, **정확하고 자연스럽게 시선을 회전시켜 비행기의 진행 방향을 따라가는 듯한 시점 전환 효과**를 제공할 수 있게 된다.

**4) Rotation Matrix 기반 Camera 회전 증명 (Rotation Only)**

A (camera.matrixWorld): game.js:1310
game.js:1311

| (색인) | 값 |
|---|---|
| 0 | 1 |
| 1 | 0 |
| 2 | 0 |
| 3 | 0 |
| 4 | 0 |
| 5 | 1 |
| 6 | 0 |
| 7 | 0 |
| 8 | 0 |
| 9 | 0 |
| 10 | 1 |
| 11 | 0 |
| 12 | 0 |
| 13 | 100 |
| 14 | 200 |
| 15 | 1 |

▸ Array(16)

**[ Rotation 전 camera 행렬 A ]**

$$A = \begin{bmatrix} 1 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & 1 \end{bmatrix}$$

R (lookAt matrix): game.js:1319
game.js:1320

| (색인) | 값 |
|---|---|
| 0 | 0.970142500145332 |
| 1 | -0 |
| 2 | 0.242535625036333 |
| 3 | 0 |
| 4 | 0 |
| 5 | 1 |
| 6 | 0 |
| 7 | 0 |
| 8 | -0.24253562503633294 |
| 9 | 0 |
| 10 | 0.9701425001453318 |
| 11 | 0 |
| 12 | 0 |
| 13 | 0 |
| 14 | 0 |
| 15 | 1 |

▸ Array(16)

**[ lookAt()으로부터 얻어진 rotation 행렬 R ]**

$$R = \begin{bmatrix} 0.9701 & 0 & 0.2425 \\ 0 & 1 & 0 \\ -0.2425 & 0 & 0.9701 \end{bmatrix}$$

B = A * R: game.js:1324

game.js:1325

| (색인) | 값 |
|---|---|
| 0 | 0.970142500145332 |
| 1 | 0 |
| 2 | 0.242535625036333 |
| 3 | 0 |
| 4 | 0 |
| 5 | 1 |
| 6 | 0 |
| 7 | 0 |
| 8 | -0.24253562503633294 |
| 9 | 0 |
| 10 | 0.9701425001453318 |
| 11 | 0 |
| 12 | -48.50712500726659 |
| 13 | 100 |
| 14 | 194.02850002906635 |
| 15 | 1 |

▸ Array(16)

[ B = A · R 행렬 계산 ]

$$B = \begin{bmatrix} 0.9701 & 0 & 0.2425 \\ 0 & 1 & 0 \\ -0.2425 & 0 & 0.9701 \end{bmatrix}$$

회전 행렬 R을 camera.matrixWorld(A)에 곱한 결과 B는, 회전 적용 후의 실제 camera의 World Transform 행렬과 일치함을 확인할 수 있었다. 이는 .lookAt()으로 생성된 회전이 정확히 적용되었음을 수학적으로 검증한 것이다.

[ 3인칭 시점에서 비행기 뒤로 카메라 옮기는 과정 ]

**[ 순서 1 ]**

**[ 순서 2 ]**

**[ 순서 3 ]**

**[ 순서 4 ]**

# 4. Attach Camera: Convert World Coordinates to Airplane Local Coordinates

Airplane 객체를 Camera의 부모로 설정하면, 이후 Camera의 Transform은 AirPlane의 L**ocal Coordinate System(로컬 좌표계)**을 기준으로 동작하게 된다.

그러나 현재까지 계산된 Camera의 위치 및 Rotation 정보는 모두 **World Coordinate System(월드 좌표계)**을 기준으로 정의되어 있기 때문에 **올바른 부모-자식 관계를 유지하면서도 시각적 변화가 생기지 않도록 하려면**, 이 World 기준 Transform을 **Airplane 기준의 Local Transform**으로 변환해줄 필요가 있다.

이를 위한 행렬 간의 관계는 다음과 같다:

- $\mathbf{T_{C0}}$: Camera의 World Transform (현재 위치/회전 상태)
- $\mathbf{T_C}$**:** Camera의 Local Transform (Airplane 기준에서 계산하고자 하는 값)
- $\mathbf{T_A}$: Airplane의 World Transform
- $\mathbf{T_A^{-1}}$**:** Airplane World Transform의 Inverse Matrix

**1) 행렬 관계 및 유도**

3D Graphics에서, 부모-자식 관계의 Transform 연산은 다음과 같은 수식을 따른다:

$$\mathbf{T_{C0}} = \mathbf{T_A} \cdot \mathbf{T_C}$$

즉, **부모(Airplane)의 World Transform과 자식(Camera)의 Local Transform**을 곱하면, **자식(Camera)의 World Transform**이 된다.

이제 $\mathbf{T_C}$를 구하기 위해 양변에 $\mathbf{T_A^{-1}}$를 곱하면:

$$\mathbf{T_A^{-1}} \cdot \mathbf{T_{C0}} = \mathbf{T_C}$$

이제 해야 할 일은 :

1. $\mathbf{T_A}$ (Airplane의 World Transform) 계산
2. $\mathbf{T_{C0}}$ (현재 Camera의 World Transform) 계산

3. $\mathbf{T_A^{-1}} \cdot \mathbf{T_{C0}} = \mathbf{T_C}$ 계산

이렇게 계산한 $\mathbf{T_C}$를 Camera에 직접 설정하면, Camera를 Airplane의 자식으로 추가한 후에도 시각적으로 동일한 위치와 방향을 유지할 수 있다.

**2) 코드 구현 설명**

**1. Attach 조건문**

```
// 전역 플래그: 카메라가 비행기의 자식으로 붙었는지 여부
let cameraAttached = false;
// 임계치: 카메라가 목표 위치와의 거리가 이 값보다 작으면 attach
const ATTACH_THRESHOLD = 4.0;
```

```
if (camera.position.distanceTo(targetPos) < ATTACH_THRESHOLD && !cameraAttached)
```

· **조건 1 :** Camera가 목표 위치에 충분히 가까워졌을 때(distanceTo로 거리 비교)

· **조건 2 :** 아직 비행기에 attach되지 않은 상태 (cameraAttached === false)

이 두 조건을 모두 만족할 때 아래 과정이 실행됨

**2. 비행기(부모 객체)와 카메라의 World Transform 저장**

```
// 비행기(부모)의 월드 트랜스폼 업데이트 후 저장
airplane.mesh.updateMatrixWorld(true);
let T_A = airplane.mesh.matrixWorld.clone();

//  현재 카메라의 월드 트랜스폼을 업데이트 후 저장
camera.updateMatrixWorld(true);
let T_C0 = camera.matrixWorld.clone();
```

이 코드는 비행기와 카메라의 matrixWorld를 업데이트한 뒤, 각각의 월드 기준 변환 행렬을 저장하는 코드이다.
이 과정을 통해, 시점 변환 직후의 카메라 위치 및 회전 정보를 월드 좌표계 기준으로 정확히 확보할 수 있다.

**3. 카메라를 비행기의 자식으로 등록**

```
//  카메라를 비행기의 자식으로 붙임
airplane.mesh.add(camera);
```

이 코드는 카메라를 airplane.mesh 객체의 자식 노드로 추가한다. 이로써 카메라의 변환 기준이 **World에서 비행기의 Local Space**로 전환된다. 하지만 이 순간 자동으로 위치가 조정되는 것은 아니기 때문에, 별도의 계산을 통해 현재의 World 기준 위치/회전 정보를 Local 기준으로 **재설정** 해줄 필요가 있다.

**4, 비행기 행렬의 역행렬 계산**

```
//  비행기 행렬(T_A)의 역행렬 계산
let inverseT_A = new THREE.Matrix4().copy(T_A).invert();
```

T_A는 비행기의 World World Transform이다. 카메라의 World Transform을 비행기 기준 Local Transform으로 바꾸기 위해 진행되어야 하는 코드다.

**5. 카메라의 Local Transform 계산**

```
let T_C = new THREE.Matrix4();
T_C.multiplyMatrices(inverseT_A, T_C0);
```

이 계산은 "현재 카메라가 월드에서 차지하고 있던 위치와 방향을, 비행기 기준에서 어떤 위치와 방향이 되어야 동일하게 보일 것인가"를 구하는 것이다.

**5. 카메라의 Local Transform 계산**

```
camera.matrix.copy(T_C);
camera.matrixAutoUpdate = false;
cameraAttached = true; // attach 완료 표시
```

camera.matrix에 직접 T_C를 대입하여 로컬 트랜스폼을 수동으로 설정한다. 이후 camera.matrixAutoUpdate = false로 설정하여, 자동 위치 갱신을 막고 직접 설정한 트랜스폼을 고정시킨다. cameraAttached = true로 플래그를 변경하여, 이 로직이 다시 반복되지 않도록 방지한다.

**[ console.log를 통해서 attach 상태를 출력 ]**

# 5. Smooth Camera Movement During View Switch and Airplane Control

비행기 조종 중 시점이 3인칭에서 1인칭으로 전환될 때, 비행기는 여전히 움직이고 회전할 수 있다. 이러한 동적인 상황에서도 카메라는 cockpit 위치로 자연스럽게 이동하고, cockpit 방향을 정확하게 바라보도록 구현해야 한다.

이를 위해 단순한 위치 전환이 아닌, **비행기의 실시간 상태를 반영한 동적인 타겟 위치 및 방향을 기반으로, 카메라의 위치와 회전 모두를 매 프레임 보간하는 로직**이 필요하다.

### 5.1 카메라의 목표 위치 및 시선 계산 (비행기 상태 반영)

```
// 1인칭 시점: cockpit 위치 계산 (비행기 로컬 좌표계를 이용)
const cockpitOffset = new THREE.Vector3(-500, 25, 10);
// 비행기 로컬 좌표계에서 월드 좌표로 변환
const cockpitWorldPos = airplane.mesh.localToWorld(cockpitOffset.clone());
targetPos.copy(cockpitWorldPos);
// 바라보는 방향도 로컬 오프셋을 이용하여 결정 (예, cockpit에서 오른쪽 100 단위)
const lookAtOffset = new THREE.Vector3(100, 0, 0);
const cockpitLookAt = airplane.mesh.localToWorld(lookAtOffset.clone());
targetLookAt.copy(cockpitLookAt);
```

cockpitOffset과 lookAtOffset은 **비행기 Local 좌표계** 상의 cockpit 위치 및 시선 방향이다.

**localToWorld()**을 통해, 위 로컬 좌표는 localToWorld()메서드를 통해 매 프레임마다 비행기의 위치와 회전 정보를 반영한 **월드 좌표계(World Space)**상의 위치로 변환된다. 즉, 비행기가 움직이면 cockpit의 월드 위치도 같이 움직이므로, 카메라가 바라볼 목표도 실시간으로 갱신된다.

### 5.2 카메라 위치와 방향을 목표에 부드럽게 따라가도록 보간 처리

```
// 부드러운 위치 전환: 현재 카메라 위치에서 목표 위치로 보간(lerp)
camera.position.lerp(targetPos, 0.03);
```

**lerp()**는 선형 보간을 수행하여, 현재 위치에서 목표 위치(targetPos)로 매 프레임 3%씩 이동하도록 한다. 이를 통해 현재 카메라 위치에서 목표 위치(targetPos)로 부드럽게 이동된다.

```
const m = new THREE.Matrix4();
m.lookAt(camera.position, targetLookAt, camera.up);
const targetQuat = new THREE.Quaternion().setFromRotationMatrix(m);
camera.quaternion.slerp(targetQuat, 0.03);
```

lookAt()을 사용하여 카메라가 목표 방향을 향하도록 하는 Rotation 행렬을 생성한다. 이 rotation 행렬은 Quaternion으로 변환되어 slerp()를 통해 부드럽게 적용된다. slerp()는 단위 구면($S^3$) 상의 가장 짧은 호를 따라 회전하기 때문에, 카메라가 매우 자연스럽게 회전한다.

이 방식은 **targetPos와 targetLookAt이 매 프레임 실시간으로 계산**되기 때문에, 비행기가 어떤

방향으로 움직이거나 회전하더라도 카메라가 그 움직임을 정확하게 추적할 수 있다.
따라서 cockpit 방향으로 이동하는 중에도 비행기의 모든 움직임을 반영하며, **끊김 없는 카메라 전환**이 가능해진다.

## 6. Fixing Disconnected Parts in Sea Geometry

Three.js의 CylinderGeometry는 기본적으로 원형 구조에서 텍스처 매핑이나 조명 계산을 위해 같은 위치에 있는 정점도 중복 생성하는 구조를 가진다. 이는 주로 **UV 좌표 또는 normal 값이 달라야 하기 때문에 의도적으로 정점이 복제**되는 것이다. 하지만 이로 인해 정점 기반 파도 애니메이션 적용 시, 동일한 위치에 있는 정점들이 **서로 다른 움직임을 가지면서 시각적인 끊어짐(seam) 현상이 발생**한다.

### 1) 기존 코드의 문제점

```
// 기본 geometry 생성
let geom = new THREE.CylinderGeometry(game.seaRadius, game.seaRadius, game.seaLength, 40, 10);
geom.rotateX(-Math.PI / 2);
// mergeVertices 적용 (복사해서 새 geometry 생성)
geom = mergeVertices(geom);
```

위와 같이 mergeVertices()를 적용했으나, CylinderGeometry내부의 **각 정점은 position외에도 uv, normal등의 속성이 서로 달라, 병합이 제대로 이루어지지 않는다.**
따라서 서로 같은 위치에 있지만 index가 다른 정점이 남게 되고, 파동 정보(amp, speed, ang)가 다르게 적용되며 시각적인 비연속이 발생한다.

### 2) 수정 방법

```
// 속성 제거 (merge 오류 방지용)
delete geom.attributes.uv;
delete geom.attributes.normal;
```

uv및 normal속성은 병합 과정에서 정점의 동일성 비교에 영향을 주기 때문에, 불필요한 속성으로 간주하고 **제거**하였다.

```
// 정점 병합
geom = mergeVertices(geom);

// 노멀 재계산
geom.computeVertexNormals();
```

position속성만 남은 상태에서 **mergeVertices()**를 적용하면, 위치가 동일한 정점들이 성공적으로 하나로 병합된다. 그리고 병합 과정에서 삭제된 normal을 보완하기 위해, **정점 병합 이후 노멀을**

**재계산**하였다.

**3) 결과 사진 (Orbitcontrols를 사용하여 멀리서 Sea의 모습을 바라본 사진)**

**[ 코드 수정 전 Sea의 끊겨있는 모습 ]**

**[ 코드 수정 후 결과 모습 ]**

## 결론

이번 과제에서는 Three.js를 기반으로 3D 공간 내에서의 시점 전환 및 객체 간 계층적 트랜스폼을 심층적으로 구현하고 분석하였다. 특히, 3인칭에서 1인칭으로의 전환 과정에서 카메라의 위치와 회전이 부드럽게 이루어지도록 .lerp()및 .slerp()를 활용하였으며, 비행기의 실시간 위치 변화를 반영하여 목표 위치와 시선을 매 프레임 재계산함으로써 자연스러운 전환을 달성하였다.

또한, World Space 좌표계 기준으로 계산된 카메라의 Transform을 Local Space 좌표계로 정확히 변환하기 위해 수학적으로 $\mathbf{T_A^{-1}} \cdot \mathbf{T_{C0}} = \mathbf{T_C}$의 행렬 연산을 적용하고, 이를 실제 Three.js 코드에서 구현해보았다. 이 과정은 객체 간 계층 구조의 핵심 개념인 Local / World Space및 Transform Composition을 깊게 이해할 수 있는 좋은 기회였다.

더불어 .lookAt()으로 생성된 회전 행렬을 수학적으로 증명하고, 그 결과가 실제 카메라의 월드 행렬과 일치함을 수식과 코드 출력으로 확인함으로써, 3D 공간 회전의 이론적 이해에 도움이 되었다.